

第3章 算法分析

3.1 引言

- 目的：预测算法行为，包括正确性分析、时间复杂性分析、空间复杂性，以判断算法是否合理、算法的优劣（一般分析问题规模较大的情况）
- 决定算法时间的因素：
 - 问题规模
 - 代码的质量（重复的调用会降低性能）
 - 计算机系统性能
 - 算法的时间复杂性
- 渐进时间复杂性：问题规模趋向于无穷大时，算法时间的变化趋势和增长速度
- 空间复杂性：运行算法需要的临时空间
 - 以时间复杂性为上界

注意：由于算法的时间一般不会随问题规模增大而减少，故本部分出现的函数默认为增函数，自变量为正数

3.2 O, Ω, Θ 表示法

- O ：若 $\exists c > 0, N > 0 : \forall n \geq N, g(n) \leq c \cdot f(n)$ ，则记作 $g(n) = O(f(n))$
 - 性质1: $\forall c > 0, a > 1, (f(n))^c = O(a^{f(n)})$
即增长速度：对数函数 $<$ 多项式函数 $<$ 指数函数
 - 性质2: 若 $f(n) = O(s(n)), g(n) = O(t(n))$ ，则
 $f(n) + g(n) = O(s(n) + t(n))$
 $f(n)g(n) = O(s(n)t(n))$
- Ω ：若 $\exists c > 0, N > 0 : \forall n \geq N, g(n) \geq c \cdot f(n)$ ，则记作 $g(n) = \Omega(f(n))$
- Θ ：若 $\exists c_1 > 0, c_2 > 0, N > 0 : \forall n \geq N, c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$ ，则记作 $g(n) = \Theta(f(n))$
- o ：若 $\forall c > 0, \exists N : \forall n \geq N, g(n) \leq c \cdot f(n)$ ，则记作 $g(n) = o(f(n))$
 - $$f(n) = o(g(n)) \Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$
 - $$f(n) = o(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge g(n) \neq O(f(n))$$
- 常用的时间复杂性：（ $\prec$ 表示增长速度前者慢于后者）

$$1 \prec \log \log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

例 $f(n+1) = _f(n)$, 举例说明填 O, Ω, Θ 的情况

解 $f(n) = n, f(n+1) = n+1$, 都可以填

$f(n) = n!, f(n+1) = (n+1)! = (n+1)f(n)$, 只能填 Ω

3.3 算法的时间

- 最优算法：如果问题 P 的复杂度为 $\Omega(f(n))$, 则能够在 $O(f(n))$ 时间内求解问题 P 的算法均为最优算法

例 若问题 P 的复杂度可以证明为 $\Omega(n^2)$, 求下列算法中可能的最优算法：

$$T_1(n) = 100n^2 + 200n + 300$$

$$T_2(n) = \frac{1}{2}n^2$$

$$T_3(n) = n + k$$

$$T_4(n) = n^3 + 10n^2 \log n + 50n^2$$

解 T_3 是错误的算法, 因为算法时间复杂度不能小于问题时间复杂度

T_1, T_2 如果正确则是最优算法, T_4 不是最优算法

- 估计算法时间的方法：
 - 计算迭代次数
 - 计算基本运算频度（注意选择合适的运算单元）
 - 分析递推关系

例 分析如下算法的时间复杂度

```

// Input: n
// Output: Count
init count = 0
while (n>=1)
    for (j = 1..n)
        count++
    endfor
    n = n/2
endwhile
return count

```

解 将 `count++` 视为最基本的运算单元，设 $2^{k-1} \leq n \leq 2^k$ ，则运算单元数 $\sum_{j=0}^k \frac{n}{2^j} = \Theta(n)$ ，所以时间复杂度 $\Theta(n)$

例 分析插入排序算法的时间复杂度（比较元素使用二分法）

解 虽然比较次数期望降低为 $O(n \log n)$ ，但元素交换次数仍为 $O(n^2)$ ，所以时间复杂度仍为 $O(n^2)$

- 分析算法的参考情况：一般以最坏情况和平均情况作为分析对象
- 输入规模：输入所占的空间，如
 - 排序问题中的数组规模 n
 - 矩阵运算中的矩阵元素个数 n^2
 - 以输入数据的大小作为输入规模时，由于机内用二进制表示，所以输入规模为 $\log n$
- XX时间算法：算法时间复杂性相对于输入规模的函数，如
 - 二分查找 $\log n$ 相对于输入规模 n 为对数时间
 - 矩阵加法 n^2 相对于输入规模 n^2 为线性时间

例 分析素数判断算法的时间

```

// Algorithm: Prime judgement
// Input: n
// Output: false/true
for i = 2 to sqrt(n)
    if (n % i == 0) return false
endfor
return true

```

解 输入规模 $O(\log n)$ ，算法时间复杂度 $O(\sqrt{n}) = O(2^{\frac{\log n}{2}})$ ，所以该算法为指数时间算法

3.4 算法分析中的常用函数及其渐进性质

3.4.1 对数函数

$$\log xy = \log x + \log y$$

$$\log x^y = y \log x$$

$$\log_a b = (\log_b a)^{-1}$$

$$\log_x y = \frac{\log_a y}{\log_a x}$$

$$a^{\log_a x} = x$$

$$x^{\log y} = y^{\log x}$$

$$\sum_{i=1}^n [\log i] = \Theta(n \log n)$$

3.4.2 顶函数和底函数

- 底函数： $\lfloor x \rfloor$ 表示不大于 x 的最大整数
- 顶函数： $\lceil x \rceil$ 表示不小于 x 的最小整数
- $x \in \mathbb{N}$, $\lceil \frac{x}{2} \rceil + \lfloor \frac{x}{2} \rfloor = x$

3.4.3 阶乘

$$n! = \Theta(\sqrt{n} \cdot n^n)$$

$$\log(n!) = \Theta(n \log n)$$

3.4.4 二项式系数

$$\binom{n}{k} = \binom{n}{n-k} = \frac{n!}{k!(n-k)!}$$
$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

3.4.5 鸽巢原理

- 将 n 个球放入 m 个盒子中，则：
 - 存在一个盒子至少装了 $\lceil \frac{n}{m} \rceil$ 个球
 - 存在一个盒子至多装了 $\lfloor \frac{n}{m} \rfloor$ 个球

例 优化以下算法，使其为有限循环

```
// Input: a,b(a<b, and both are positive integers)
// Output: a/b(0.xxx...)
Output("0.")
a = a*10
while (a>0)
    Output(a/b)
    a = (a%b)/10
endwhile
```

解 由于 $a \% b$ 最多有 b 种取值，则由鸽巢原理， b 次循环内必能找到循环体，添加循环次数不大于 b 的限制即可

3.5 和式

3.5.1 渐进性质

$$\begin{aligned}\sum_{j=1}^n j &= \frac{n(n+1)}{2} = \Theta(n^2) \\ \sum_{j=1}^n j^2 &= \frac{n(n+1)(n+2)}{6} = \Theta(n^3) \\ \sum_{j=1}^n c^j &= \frac{c^{n+1}-1}{c-1} = \Theta(c^n) \\ \sum_{j=1}^n jc^j &= \frac{nc^{n+2}-nc^{n+1}-c^{n+1}+c}{(c-1)^2} = \Theta(nc^n) \\ \sum_{j=1}^n \frac{1}{j} &= \Theta(\log n) \\ \sum_{j=1}^n \frac{1}{c^j} &= \Theta(1) \\ \sum_{j=1}^n \frac{j}{c^j} &= \Theta(1)\end{aligned}$$

其中 c 为常数且 $c > 1$

3.5.2 积分近似

- 对于单调递增函数 $f(n)$ ，有

$$\int_{m-1}^n f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x)dx$$

例 证明 $\log(n!) = \Theta(n \log n)$

解 $\log(n!) = \sum_{i=1}^n \log i$

$$\begin{aligned} \sum_{i=1}^n \log i &= \log n + \sum_{i=1}^{n-1} \log i \\ &\leq \log n + \int_1^n \log x dx \\ &= \log n + n \log n + n \log e + \log e \\ &= O(n \log n) \end{aligned}$$

$$\begin{aligned} \sum_{i=1}^n \log i &= \log 1 + \sum_{i=2}^n \log i \\ &\geq 0 + \int_1^n \log x dx \\ &= n \log n + n \log e + \log e \\ &= \Omega(n \log n) \end{aligned}$$

$\therefore \log(n!) = \Theta(n \log n)$

3.6 递推关系

- 递推关系：函数的定义体中包含函数本身

3.6.1 合理猜测

例 $T(2n) \leq 2T(n) + 2n - 1$, $T(2) = 1$, 求 $T(n)$ 的渐进复杂度

解 不妨令 $n = 2^k$

1. 设 $T(n) \leq c \cdot n^2$
 $T(2) = 1 \leq 4c$ 成立

$$T(2n) \leq 2T(n) + 2n - 1 \leq 2cn^2 + 2n - 1 \leq c \cdot (2n)^2 \text{ 成立}$$

$$\therefore T(n) = O(n^2)$$

2. 设 $T(n) \leq c \cdot n$

$$T(2) = 1 \leq c \text{ 成立}$$

$$T(2n) \leq 2T(n) + 2n - 1 \leq 2cn + 4n - 1 \leq c \cdot 2n \text{ 不成立}$$

$$\therefore n = o(T(n))$$

3. 设 $T(n) \leq c \cdot n \log n$

$$T(2) \leq c \cdot 2 \log 2 \text{ 成立}$$

$$T(2n) \leq 2T(n) + 2n - 1 \leq 2cn \log n + 2n \log 2 - 1 = 2n \log 2n \text{ 成立}$$

$$\therefore T(n) = O(n \log n)$$

结论: $T(n) = O(n \log n)$

- 线性齐次递推式：下式

$$f(n) = \sum_{i=1}^k a_i f(n-i)$$

称为 $f(n)$ 的 k 阶线性齐次递推式

$$\circ k=1, f(n) = af(n-1) = a^2 f(n-2) = \dots = a^n f(0)$$

$$\circ k=2, f(n) = a_1 f(n-1) + a_2 f(n-2) \Rightarrow x^2 = a_1 x + a_2, \text{ 解特征方程 } x^2 = a_1 x + a_2 \text{ 得到两个根 } r_1, r_2$$

- 若 $r_1 \neq r_2$, 可设 $f(n) = c_1 r_1^n + c_2 r_2^n$
- 若 $r_1 = r_2$, 可设 $f(n) = c_1 r^n + c_2 n r^n$

3.6.2 分治关系

- $T(n) = aT(\frac{n}{b}) + cn^k$ 在 $a \geq 1, b \geq 2$ 时的解为:

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

- 证明: 设 $T(1) = c, n = b^m$, 则有

$$\begin{aligned}
T(n) &= a(T(\frac{n}{b})) + cn^k \\
&= a^2T(\frac{n}{b^2}) + ac(\frac{n}{b})^k + cn^k \\
&= \dots \\
&= c \sum_{i=0}^m a^{m-i} b^{ik} \\
&= ca^m \sum_{i=0}^m (\frac{b^k}{a})^i
\end{aligned}$$

当 $a > b^k$ 时

$$\begin{aligned}
&\sum_{i=0}^{\infty} (\frac{b^k}{a})^i = Constant \\
T(n) &= O(a^m) = O(a^{\log_b n}) = O(n^{\log_a b})
\end{aligned}$$

当 $a = b^k$ 时

$$\begin{aligned}
&\sum_{i=0}^m (\frac{b^k}{a})^i = m \\
T(n) &= O(m \cdot a^m) = O(n^k \log n)
\end{aligned}$$

当 $a < b^k$ 时，令

$$F = \frac{b^k}{a}$$

则

$$T(n) = c \cdot a^m \frac{F^{m+1} - 1}{F - 1} = O((aF)^m) = O(b^{km}) = O(n^k)$$

- 主定理：若

$$T(n) = aT(\frac{n}{c}) + f(n)$$

， 则

$$T(n) = \begin{cases} \Theta(n^{\log_c a}) & \exists \epsilon > 0 : f(n) = O(n^{\log_c a - \epsilon}) \\ \Theta(n^{\log_c a} \cdot (\log n)^{k+1}) & \exists k > 0 : f(n) = \Theta(n^{\log_c a} \cdot (\log n)^k) \\ \Theta(f(n)) & \begin{aligned} &\exists \epsilon > 0 : f(n) = \Omega(n^{\log_c a + \epsilon}) \\ &\wedge \exists k < 1 : af(\frac{n}{c}) \leq kf(n) \end{aligned} \end{cases}$$

- 注：主定理不能完全覆盖所有情况

例 求 $T(n)$ 的复杂度

$$(1) T(n) = 5T\left(\frac{n}{2}\right) + n^2$$

$$(2) T(n) = 27T\left(\frac{n}{3}\right) + n^3 \log n$$

$$(3) T(n) = 5T\left(\frac{n}{2}\right) + n^3$$

$$(4) T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

解 套用主定理即可

$$(1) f(n) = n^2 \Rightarrow \exists \epsilon > 0 : f(n) = O(n^{\log_2 5 - \epsilon})$$

$$\therefore T(n) = \Theta(n^{\log_2 5})$$

$$(2) f(n) = n^3 \log n \Rightarrow \exists k = 1 > 0 : f(n) = \Theta(n^{\log_3 27} \cdot (\log n)^k)$$

$$\therefore T(n) = \Theta(n^3 \log^2 n)$$

$$(3) f(n) = n^3 \Rightarrow \exists \epsilon > 0 : f(n) = O(n^{\log_2 5 + \epsilon})$$

$$af\left(\frac{n}{2}\right) = \frac{5}{8}n^3 \Rightarrow \exists k = \frac{5}{8} < 1 : af\left(\frac{n}{c}\right) \leq kf(n)$$

$$\therefore T(n) = \Theta(n^3)$$

$$(4) f(n) = \frac{n}{\log n}, n^{\log_c a} = n, f(n) = o(n)$$

不满足主定理的任意条件，不可推断

3.6.3 涉及全部历史的递推关系

- 定义：依赖于先前所有函数值的函数
- 方法：历史消除法

例 求 $T(n)$ 的复杂度

$$(1) T(n) = c + \sum_{i=1}^{n-1} T(i), n \geq 2$$

$$(2) T(n) = n - 1 + \frac{2}{n} \sum_{i=1}^{n-1} T(i), n \geq 2$$

解 历史消除法

$$(1) T(2) = c + T(1) = O(1)$$

$$T(n-1) = c + \sum_{i=1}^{n-2} T(i)$$

$$T(n) - T(n-1) = T(n-1)$$

$$T(n) = 2T(n-1) = \dots = 2^{n-2}T(2) = O(2^n)$$

(2)

$$\begin{aligned}
T(n+1) &= n + \frac{2}{n+1} \sum_{i=1}^n T(i) \\
T(n+1) &= \frac{n+2}{n+1} T(n) + \frac{2n}{n+1} \\
&\leq \frac{n+2}{n+1} T(n) + 2 \\
&\leq \frac{n+2}{n+1} \left(\frac{n+1}{n} T(n) + 2 \right) + 2 \\
&\leq \dots \\
&\leq 2n \left(\frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{3} \right)
\end{aligned}$$

$$\therefore T(n) = O(n \log n)$$